

Diseño de Datos y Algoritmos

Estrategia incremental

Ordenamiento · Insertion Sort · Invariantes ·
Complejidad asintótica

2026-2



```
for j = 2 to A.length
  key = A[j]
  i = j - 1
  while i > 0 and A[i] > key:
    A[i+1] = A[i]
    i = i - 1
  A[i+1] = key
```

**Construir soluciones correctas a partir de un
invariante: la esencia de la estrategia incremental.**

Agenda de trabajo

De la especificación del ordenamiento al análisis asintótico.

- | | | |
|-----------|-------------------------------------|---|
| 01 | Problema de ordenamiento | Especificación y criterio de corrección. |
| 02 | Diseño con Insertion Sort | Intuición, invariante y pseudocódigo. |
| 03 | Análisis de complejidad | RAM, mejor caso y peor caso. |
| 04 | Código y notación asintótica | Python, Θ , O , Ω y orden de crecimiento. |
| 05 | Ejercicios | Aplicar y comparar funciones. |

01

BLOQUE 1

Problema de ordenamiento

Qué significa ordenar correctamente y cómo comienza el razonamiento algorítmico.

Especificación

Definir con claridad la entrada, la salida y el criterio de corrección.

Entrada Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Salida Una permutación $\langle a'_1, a'_2, \dots, a'_n \rangle$ tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Algoritmo correcto

Un algoritmo se dice correcto si, para cada instancia válida de entrada, produce una salida correcta.

Arreglo de entrada					
1	2	3	4	5	6
5	2	4	6	1	3

Idea clave

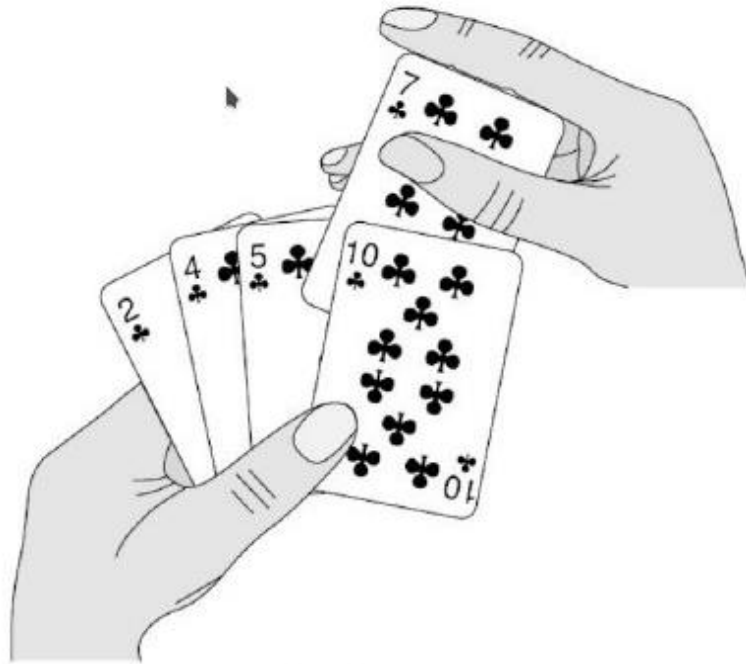
Ordenar no cambia los elementos.

Ordenar solo cambia su posición.

La corrección exige preservar y ordenar.

Insertion Sort

La estrategia incremental inserta cada elemento en la parte ya ordenada.



Insertion Sort organiza cada elemento donde corresponde, igual que una persona ordena una mano de cartas.

Insertion Sort

Identificar la condición que permanece verdadera en cada iteración.

Arreglo de entrada					
1	2	3	4	5	6
5	2	4	6	1	3

¿Qué condición se cumple siempre?

Insertion Sort

La parte izquierda del arreglo crece ordenada en cada iteración.



Paso (a)

Antes del actual, el prefijo [5] ya está ordenado.

Los elementos antes del actual son los originales.

Además, se mantienen ordenados.

Insertion Sort

La parte izquierda del arreglo crece ordenada en cada iteración.



Paso (b)

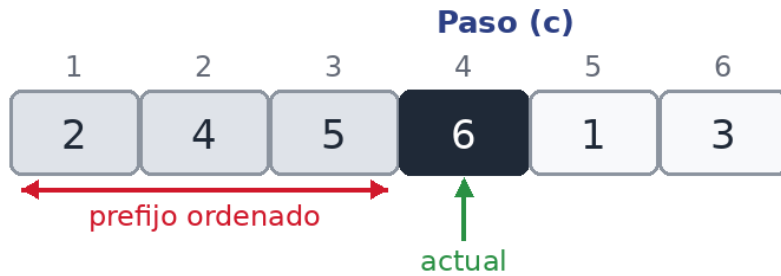
Después de insertar 4, el prefijo [2,5] se mantiene ordenado.

Los elementos antes del actual son los originales.

Además, se mantienen ordenados.

Insertion Sort

La parte izquierda del arreglo crece ordenada en cada iteración.



Paso (c)

El elemento 6 entra al final sin romper el orden.

Los elementos antes del actual son los originales.

Además, se mantienen ordenados.

Insertion Sort

La parte izquierda del arreglo crece ordenada en cada iteración.



Paso (d)

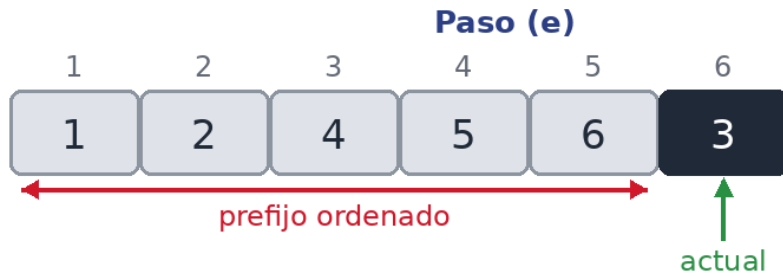
El 1 debe desplazarse hasta la primera posición.

Los elementos antes del actual son los originales.

Además, se mantienen ordenados.

Insertion Sort

La parte izquierda del arreglo crece ordenada en cada iteración.



Paso (e)

Al insertar 3, el prefijo [1,2,4,5,6] queda ordenado.

Los elementos antes del actual son los originales.

Además, se mantienen ordenados.

El invariante de Insertion Sort

Ahora la propiedad ya se puede expresar con precisión.



Invariante

Los elementos antes del actual:

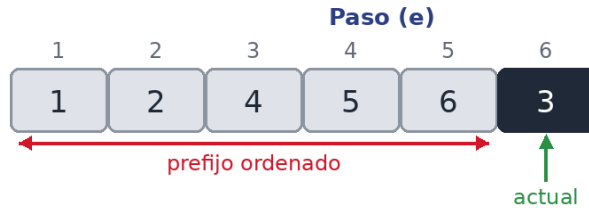
son los originales

y ahora están ordenados

Esta propiedad permite probar la corrección.

Insertion Sort

El pseudocódigo muestra explícitamente la inserción dentro del prefijo ordenado.



Cada iteración toma $A[j]$ y la ubica donde corresponde.

```
INSERTION-SORT(A)
```

```
1  for  $j = 2$  to  $A.length$ 
```

```
2       $key = A[j]$ 
```

```
3      // Insert  $A[j]$  in sorted sequence  $A[1..j-1]$ 
```

```
4       $i = j - 1$ 
```

```
5      while  $i > 0$  and  $A[i] > key$ 
```

```
6           $A[i + 1] = A[i]$ 
```

```
7           $i = i - 1$ 
```

```
8       $A[i + 1] = key$ 
```

Invariante

Para demostrar la corrección de un ciclo se analizan tres momentos.

1

Iniciación

El invariante es verdadero antes de la primera iteración.

2

Estabilidad

Si es verdadero antes de una iteración, sigue siéndolo después.

3

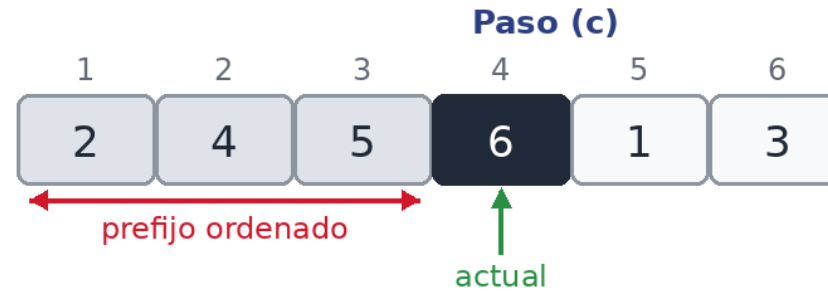
Terminación

Al terminar el ciclo, el invariante lleva a la corrección.

Insertion Sort

Formulación explícita del invariante del ciclo externo.

```
for j = 2 to A.length
  key = A[j]
  // Insert A[j] into
  // sorted sequence
  A[1..j-1]
  i = j - 1
  while i > 0 and A[i] >
  key
    A[i+1] = A[i]
    i = i - 1
  A[i+1] = key
```



$A[1 .. j - 1]$ consta de los elementos originalmente en $A[1 .. j - 1]$, pero ordenados ascendentemente.

Iniciación

Verificar el invariante antes de comenzar la primera iteración.

```
for j = 2 to A.length  
    ...
```



Se debe demostrar que el invariante es cierto antes de la primera iteración, es decir, cuando $j = 2$.

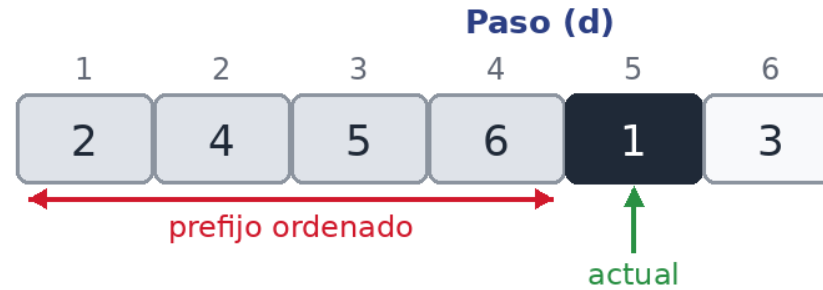
El subarreglo $A[1..j-1]$ contiene solo $A[1]$.

Un solo elemento está ordenado de manera trivial.

Estabilidad

Suponer que el invariante vale al inicio de la iteración y probar que se conserva.

```
for j = 2 to A.length
  key = A[j]
  i = j - 1
  while i > 0 and A[i] > key
    A[i + 1] = A[i]
    i = i - 1
  A[i + 1] = key
```



Antes de ejecutar el ciclo, $A[1..j-1]$ está ordenado.

El while desplaza a la derecha los elementos mayores que key.

Al insertar key en su posición, $A[1..j]$ queda ordenado.

Terminación

Al finalizar el ciclo, el invariante demuestra la corrección global.

```
for j = 2 to A.length  
    ...  
# termina cuando j = A.length
```

Resultado final

1	2	3	4	5	6
1	2	3	4	5	6

Cuando el ciclo termina, $j = n + 1$.

El invariante garantiza que $A[1..n]$ está ordenado ascendentemente.

Como además contiene los valores originales, el algoritmo es correcto.

Análisis de complejidad

Medir los recursos que necesita un algoritmo para resolver un problema.

El análisis de algoritmos identifica recursos como tiempo de ejecución y memoria.

Usaremos el modelo RAM (Random Access Machine).

Un solo procesador.

Instrucciones secuenciales.

Operaciones aritméticas y booleanas con costo constante.



Modelo simple para estimar costo temporal y espacial.

Análisis de complejidad

Generalmente estudiamos el tiempo de ejecución como una función del tamaño de la entrada.



Tamaño de la entrada: depende del problema y mide la cantidad de elementos de entrada.

Tiempo de ejecución: número de operaciones primitivas o pasos necesarios para resolver el problema.

Nos interesa cómo crece ese tiempo cuando n aumenta.

Análisis de complejidad

El foco suele estar en el peor caso, aunque también observamos mejor y promedio.

Estamos interesados en encontrar el tiempo de ejecución más largo.

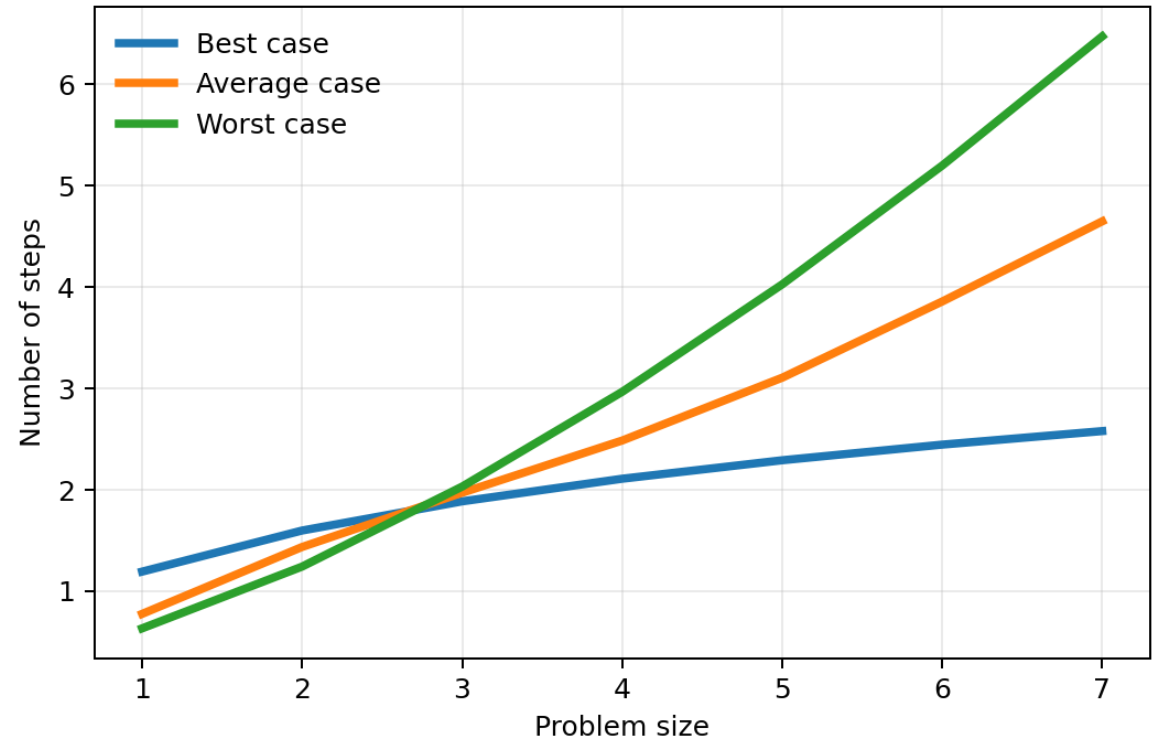
Un límite superior describe el comportamiento del algoritmo para cualquier entrada.

Muchas veces el peor caso ocurre con frecuencia o facilita comparaciones seguras.

BEST

AVG

WORST



Descomposición de costo

Cada línea del algoritmo aporta un costo y un número de ejecuciones.

Pseudocódigo	Costo	Veces
1 for j = 2 to A.length	c_1	n
2 key = A[j]	c_2	$n - 1$
3 // Insert A[j] in sorted A[1..j-1]	0	$n - 1$
4 i = j - 1	c_4	$n - 1$
5 while i > 0 and A[i] > key	c_5	$\sum t_j$
6 A[i + 1] = A[i]	c_6	$\sum (t_j - 1)$
7 i = i - 1	c_7	$\sum (t_j - 1)$
8 A[i + 1] = key	c_8	$n - 1$

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5\sum t_j + c_6\sum(t_j-1) + c_7\sum(t_j-1) + c_8(n-1)$$

InsertionSort: $T(n)$

La expresión se simplifica según el comportamiento de t_j .

Mejor caso

Si el arreglo ya está ordenado, entonces $t_j = 1$ para $j = 2, 3, \dots, n$.

$$T(n) = an + b.$$

Es una función lineal de n .

Peor caso

Si el arreglo está en orden descendente, entonces $t_j = j$ para $j = 2, 3, \dots, n$.

$$T(n) = an^2 + bn + c.$$

Es una función cuadrática de n .

Insertion Sort tiene mejor caso $\Theta(n)$ y peor caso $\Theta(n^2)$.

Código

Versión en Python de Insertion Sort.

```
def insertion_sort(A):  
    for j in range(1, len(A)):  
        key = A[j]  
        i = j - 1  
  
        while i >= 0 and A[i] > key:  
            A[i + 1] = A[i]  
            i = i - 1  
  
        A[i + 1] = key
```

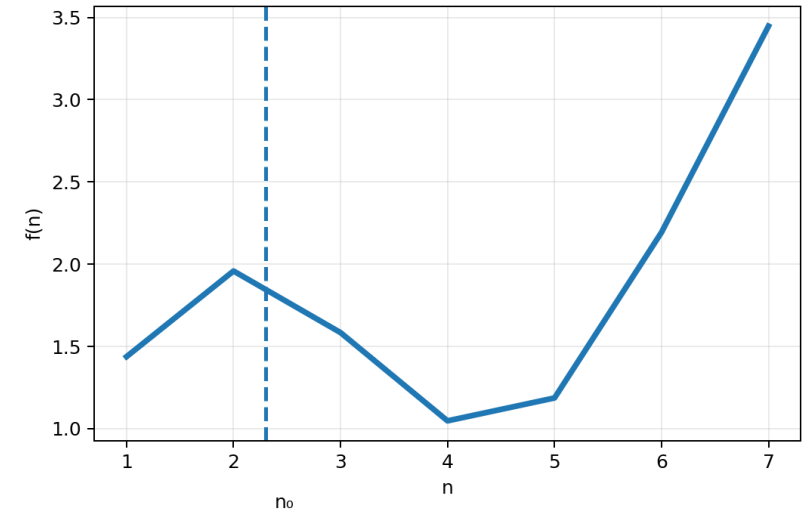

Análisis de complejidad asintótico

Nos interesa cómo crece el tiempo cuando el tamaño de la entrada aumenta sin límite.

El orden de crecimiento caracteriza la eficiencia de manera simple.

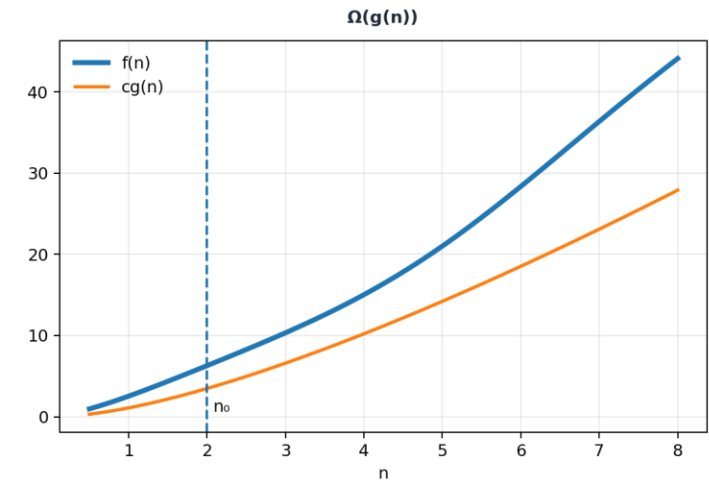
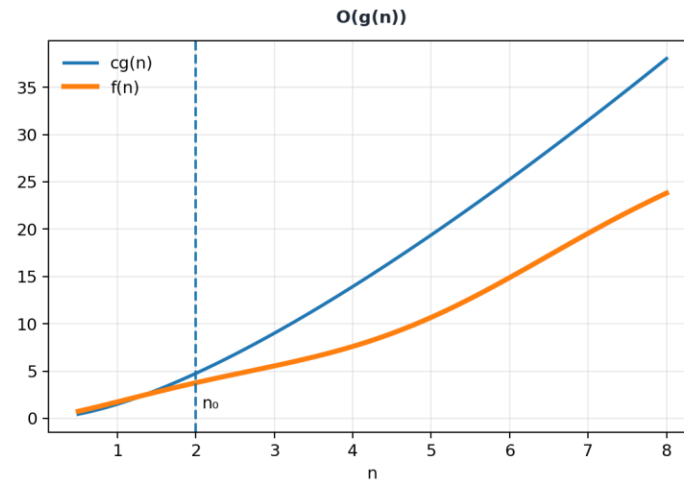
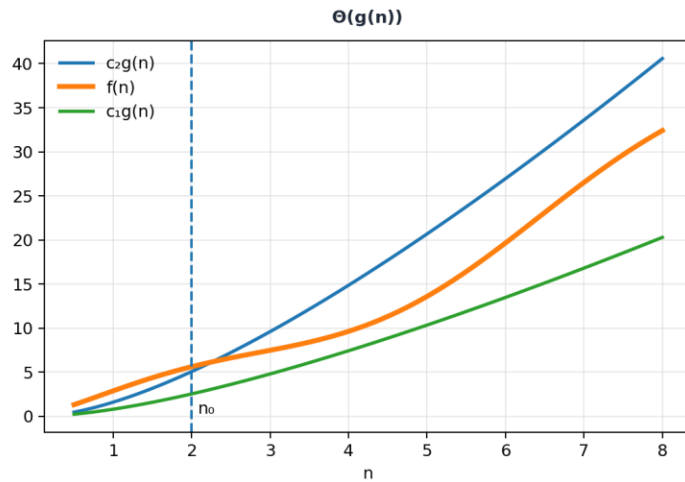
La notación asintótica permite comparar algoritmos independientemente de detalles constantes.

Trabajamos con funciones cuyo dominio son los naturales $n \in \{0, 1, 2, \dots\}$.



Notaciones Θ , O y Ω

Cada notación expresa una forma distinta de acotar el crecimiento.



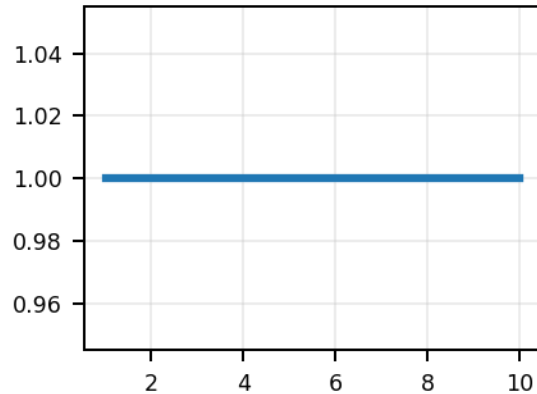
Θ acota por arriba y por abajo · O establece un límite superior · Ω establece un límite inferior.

Orden de crecimiento

Algunas familias de funciones aparecen con frecuencia en el análisis de algoritmos.

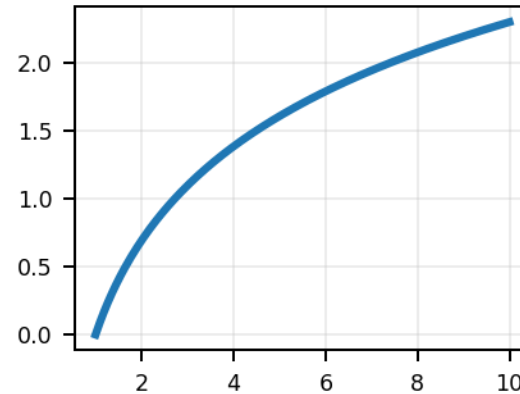
$$1 \ll \log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n!$$

Constante



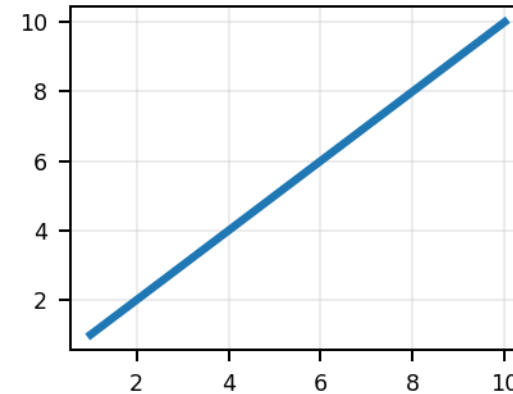
$$f(n)=1$$

Logarítmico



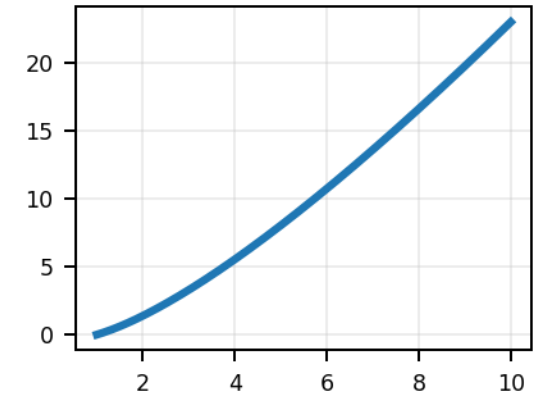
$$f(n)=\log n$$

Lineal



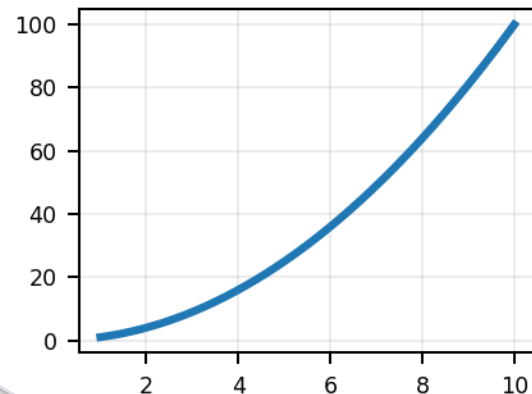
$$f(n)=n$$

Superlineal



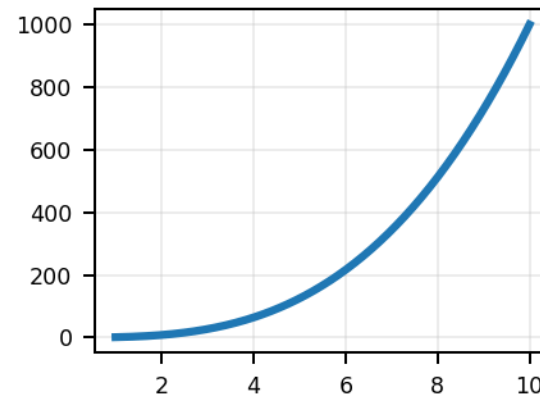
$$f(n)=n \log n$$

Cuadrático



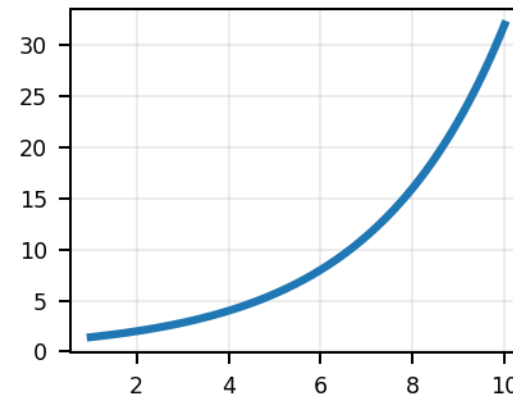
$$f(n)=n^2$$

Cúbico



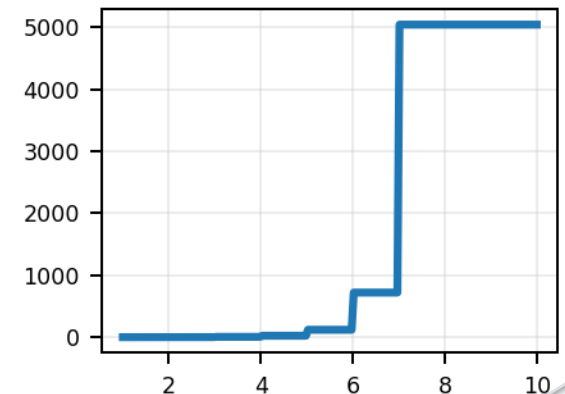
$$f(n)=n^3$$

Exponencial



$$f(n)=c^n$$

Factorial



$$f(n)=n!$$

Orden de crecimiento

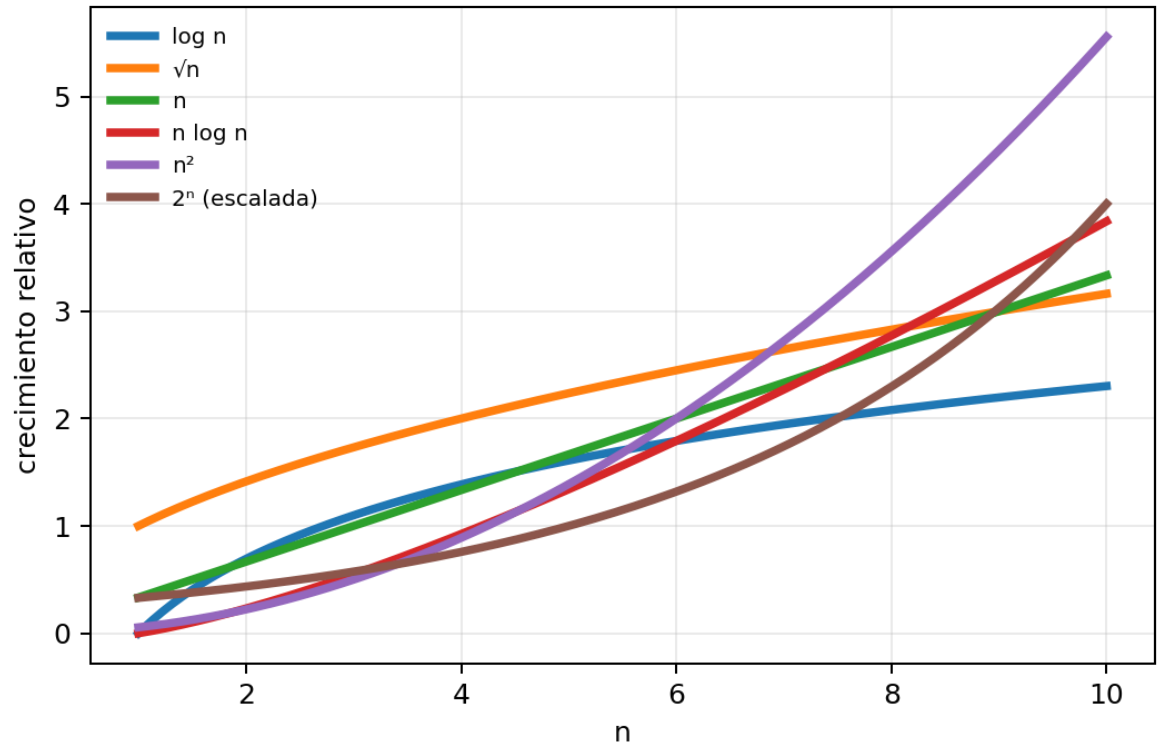
Comparar gráficamente ayuda a entender la separación entre familias.

Relaciones útiles

$$\log n < \sqrt{n} < n < n \log n < n^2 < 2^n$$

Los algoritmos polinomiales suelen escalar mejor que los exponenciales.

El crecimiento domina a las constantes para n suficientemente grande.



Análisis comparativo

Las tablas hacen tangible la diferencia entre órdenes de crecimiento.

Valores para distintos n

n	log n	n	n log n	n ²	n ³	2 ⁿ
8	3	8	24	64	512	256
16	4	16	64	256	4096	65536
32	5	32	160	1024	32768	4294967296
64	6	64	384	4096	262144	1.84e19
128	7	128	896	16384	2097152	3.40e38
256	8	256	2048	65536	16777216	1.15e77
512	9	512	4608	262144	134217728	1.34e154

Máximo tamaño resoluble (ejemplo)

Running time	1 second	1 minute	1 hour
400n	2,500	150,000	9,000,000
2n ²	707	5,477	42,426
2 ⁿ	19	25	31

La diferencia entre crecimiento lineal, cuadrático y exponencial impacta radicalmente el tamaño de problema que puede resolverse.

Ejercicios

Aplicar los conceptos de orden de crecimiento e invariante.

1 Ordenar las siguientes funciones de menor a mayor orden:

n , $n - n^3 + 7n^5$, $n^2 + \log n$, n^3 , 2^n , $\log n$, n^2 , $(\log n)^2$, $n \log n$, $\sqrt{n}$, 2^{n-1} , $n!$, $\ln n$, e^n , $\log \log n$, $n^{1+\varepsilon}$ ($0 < \varepsilon < 1$)

2 Para las siguientes funciones, determinar el peor caso usando notación Big Oh:

mystery(n)

```
r := 0
for i := 1 to n-1 do
  for j := i+1 to n do
    for k := 1 to j do
      r := r + 1
return(r)
```

pesky(n)

```
r := 0
for i := 1 to n do
  for j := 1 to i do
    for k := j to i+j do
      r := r + 1
return(r)
```

prestiferous(n)

```
r := 0
for i := 1 to n do
  for j := 1 to i do
    for k := j to i+j do
      for l := 1 to i+j-k do
        r := r + 1
return(r)
```

3 Implementar Insertion Sort para ordenar en orden descendente en vez de ascendente.

Ejercicios

Aplicar los conceptos de orden de crecimiento e invariante.

1 Ordenar las siguientes funciones de menor a mayor orden:

n , $n - n^3 + 7n^5$, $n^2 + \log n$, n^3 , 2^n , $\log n$, n^2 , $(\log n)^2$, $n \log n$, $\sqrt{n}$, 2^{n-1} , $n!$, $\ln n$, e^n , $\log \log n$, $n^{1+\varepsilon}$ ($0 < \varepsilon < 1$)

2 Para las siguientes funciones, determinar el peor caso usando notación Big Oh:

mystery(n)

```
r := 0
for i := 1 to n-1 do
  for j := i+1 to n do
    for k := 1 to j do
      r := r + 1
return(r)
```

pesky(n)

```
r := 0
for i := 1 to n do
  for j := 1 to i do
    for k := j to i+j do
      r := r + 1
return(r)
```

prestiferous(n)

```
r := 0
for i := 1 to n do
  for j := 1 to i do
    for k := j to i+j do
      for l := 1 to i+j-k do
        r := r + 1
return(r)
```

3 Implementar Insertion Sort para ordenar en orden descendente en vez de ascendente.